

COMPUTER SCIENCE MENTORS 61A

September 23 – September 26, 2025

Example Timeline

- Ice Breaker [5 min]

If you want, start off with an icebreaker of your choice!

Also, there is a link to a feedback form at the bottom of this worksheet! It would be amazing if you could highlight this/fill it out yourself so we can improve the worksheets in the future!

- Mini-lecture: Recursion and Tree Recursion [7 min]
- Q1: Num Digits [7 min]
- Q2: Recursion Environment Diagram [10 min]
- Q3: Collapse [10 min]
- Q4: Gibonacci [7 min]
- Q5: Donut [12 min]
- Q6: Midterm [If time]

1 Recursion

1. Complete the definition for `num_digits`, which takes in a number n and returns the number of digits it has. Implement the function recursively.

```
def num_digits(n):  
    """Takes in an positive integer and returns the number of  
    digits.  
  
    >>> num_digits(0)  
    1  
    >>> num_digits(1)  
    1  
    >>> num_digits(7)  
    1  
    >>> num_digits(1093)  
    4  
    """  
  
    if n < 10:  
        return 1  
    else:  
        return 1 + num_digits(n // 10)
```

Suggested Time: 7 min

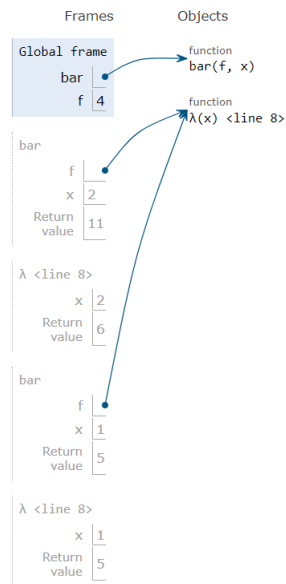
- Make sure to emphasize the structure of recursion, and the base case and recursive case in this problem.
- It may be helpful to go through the doctests and explain how the recursion works.

2. Draw the environment diagram that results from running the code.

```
def bar(f, x):  
    if x == 1:  
        return f(x)  
    else:  
        return f(x) + bar(f, x - 1)
```

```
f = 4  
bar(lambda x: x + f, 2)
```

Output: 3



<https://goo.gl/kR2n9M>

Suggested Time: 10 min

- Feel free to take extra time on this if needed!

3. Fill in `collapse`, which takes in a non-negative integer `n` and returns the number resulting from removing consecutive repeating digits, i.e. while the number may have digits repeating elsewhere in the number, the same number cannot appear next to itself.

```
def collapse(n):  
    """  
    >>> collapse(9)  
    9  
    >>> collapse(12)  
    12  
    >>> collapse(22)  
    2  
    >>> collapse(12234441)  
    12341  
    >>> collapse(11200000013333)  
    12013  
    """  
    rest, last = n // 10, n % 10  
  
    if _____:  
        _____  
  
    elif _____:  
        _____  
  
    else:  
        _____
```

```
def collapse(n):  
    rest, last = n // 10, n % 10  
    if rest == 0:  
        return last  
    elif last == rest % 10:  
        return collapse(rest)  
    else:  
        return collapse(rest) * 10 + last
```

Teaching Tips

- Demonstrating with the doc tests is very important- the problem description can be confusing.
- How are we going to traverse the number?
 - As always, we traverse right to left, since traversing left to right will only produce the same results for more work.
 - Knowing this, we can figure `rest` stands for the remaining number and `last` stands for the last digit
- What are the base cases?
 - The simplest possible case is if `n` is one digit, at which there is nothing to compare it to.
- What are the recursive cases?
 - Either we remove a digit or we do not.
 - How do we structure the recursive call when we **don't** want to get rid of `last`? We need to add it back on **after** the recursive call.
- Why does this work?
 - Remind students that part of the recursive leap of faith is to trust that calling `collapse(left)` will remove identical left adjacent numbers.
 - All they need to care about in the moment is whether or not `last` should be removed or not.

Suggested Time: 10 min

- This is a little bit harder than the num digits question, but builds upon the structure of performing arithmetic on a recursive call. A good hint is to ask students how we can skip a digit in the number.

2 Tree Recursion

1. The *Gibonacci sequence* is a recursively defined sequence of integers; we denote the n th Gibonacci number g_n . The first three terms of the sequence are $g_0 = 0, g_1 = 1, g_2 = 2$. For $n \geq 3$, g_n is defined as the sum of the previous three terms in the sequence.

Complete the function `gib`, which takes in an integer `n` and returns the n th Gibonacci number, g_n . Also, identify the three parts of recursive function design as they are used in your solution.

```
def gib(n):  
    """  
    >>> gib(0)  
    0  
    >>> gib(1)  
    1  
    >>> gib(2)  
    2  
    >>> gib(3) # gib(2) + gib(1) + gib(0) = 3  
    3  
    >>> gib(4) # gib(3) + gib(2) + gib(1) = 6  
    6  
    """  
    if _____:  
        return _____  
  
    return _____
```

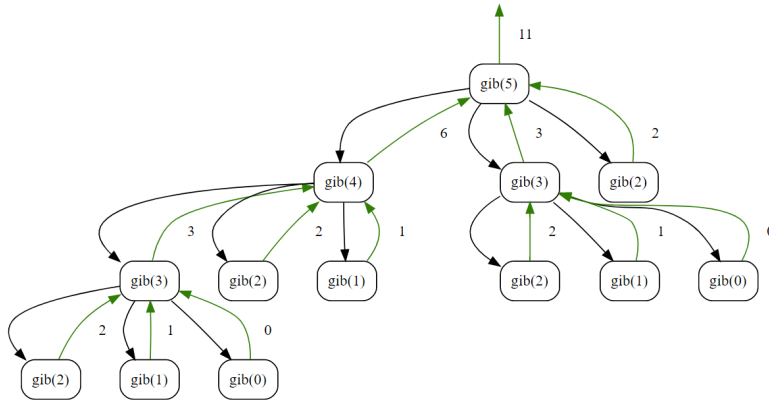
```
def gib(n):  
    if n <= 2:  
        return n  
    return gib(n - 1) + gib(n - 2) + gib(n - 3)
```

- Base case: `if n <= 2: return n`
- Recursive calls: `gib(n - 1), gib(n - 2), gib(n - 3)`
- Solving the larger problem: adding the results of the three recursive calls.
- This problem is very similar to Fibonacci, which students should be familiar with from lecture.
- Ensure that students understand how the minor alteration to the sequence slightly modifies the recursive calls and the base case

2. Including the original call, how many calls are made to `gib` when you evaluate `gib(5)`?

13.

There are a few ways you could determine this. First, you could draw out a call graph that shows all of the calls that are made. <https://tinyurl.com/gibsol>



You could also build things from the ground up using recursive intuition.

- `gib(0)` takes one call because it's a base case.
- `gib(1)` takes one call because it's a base case.
- `gib(2)` takes one call because it's a base case.
- `gib(3)` takes four calls: one for the original call, one for `gib(0)`, one for `gib(1)`, and one for `gib(2)`.
- `gib(4)` takes seven calls: one for the original call, four for `gib(3)`, one for `gib(2)`, and one for `gib(1)`.
- `gib(5)` takes 13 calls: one for the original call, seven for `gib(4)`, four for `gib(3)`, and one for `gib(2)`.

This second method is much faster, especially for higher values of `n`.

In general, I believe that drawing out call graphs can be confusing for students, especially for more difficult recursive problems. The whole idea behind recursion, after all, is that you don't have to worry about the dense tree of calls that result from a single call to a recursive function—you can just trust that the recursive calls you do make return the correct result and don't have to worry about how they specifically got that result. However, I believe that it is useful to do this sort of thing once because it demonstrates how recursion actually works in practice and why the recursion is valid and useful.

When teaching this problem, I recommend first going over the call tree. I would emphasize a few ideas. First, our base cases are at the bottom of the call graph, and every call eventually terminates at one of the base cases. They literally serve as the foundation of the tree. They are the "truth" upon which the function's validity for higher inputs rests. I would also point out that every call that's not a base case has three calls underneath it because each of these calls makes three recursive calls. Hopefully this makes the idea of recursion more concrete for the students and quells perennial concerns that the logic behind recursion is circular.

I would then going over the "smarter" way of doing the problem. I would point out how doing the problem in this way allows us to avoid the repetitive, confusing, and error prone work of considering

every call in the function. I would note that if we tried to do the same thing with `gib(1000)`, the graph would be untenable. The lesson I want them to walk away with is: abstracting away the huge number of calls is a valuable tool when working with recursion, and that they should try to avoid thinking about these calls if at all possible, as it only gets more confusing for more complicated functions.

Suggested Time: 7 min

- It may help to go over the fibonacci example for tree recursion.

3. Gabe's Donut shop has an unlimited supply of f different flavors of donuts. Adit wants to buy a box containing d donuts. Complete the skeleton for the function `donut`, which determines the number of possible ways there are for Adit to select his d donuts from the f flavors. You may assume that d and f are non-negative integers.

Hint: Does order matter?

```
def donut(d, f):
    """
    >>> donut(12, 1)
    1
    >>> donut(12, 2)
    13
    >>> donut(12, 12)
    1352078
    >>> donut(0, 0)
    1
    """
    if _____:
        return _____

    if _____:
        return _____

    return _____

def donut(d, f):
    if d == 0:
        return 1
    if f == 0:
        return 0
    return donut(d - 1, f) + donut(d, f - 1)
```

Suppose the flavors are numbered 1 through f . Because order does not matter, we're going to say that Adit fills up his donut box starting with flavor number f and working down to flavor number 1. Under this scheme, Adit has a choice to make: does he want a donut of flavor number f , or does he not want one? If he does initially take a donut of flavor number f , then the number of ways to select the remaining $d - 1$ slots of the box is `donut(d - 1, f)`. If he does not want a donut of flavor number f , then he has d slots left in the box and $f - 1$ flavors to fill them with, making the number of ways to make the selection `donut(d, f - 1)`. Since he must make the choice one way or the other, the total number of ways to fill up the box is `donut(d - 1, f) + donut(d, f - 1)`.

If your students are confused on why the doctest `donut(0, 0)` returns 1, you can think about it this way:

if you want 0 donuts and they have no flavors, then there is one action you can take, do nothing and don't purchase donuts

A big concept within tree recursion counting problems is this idea that we can use some sort of "choice" to break our problem down into smaller subproblems. I would emphasize this heavily to my students. Here, the choice is whether or not Adit takes a certain flavor.

In this case, we have deliberately neglected to provide several base cases in the doc tests. While a good exam problem would include these doctests, they are often not provided, and students should be able to deduce the bases cases by either 1) mathematical maturity or 2) deduction from test inputs. In the latter case, they could for example note that `donut(1, 1) == donut(0, 1) + donut(1, 0)` and use this to help figure out what `donut` should return at these base cases.

Suggested Time: 12 min

- Walk through some small examples and how we can visualize the different combinations of donuts!
4. Imagine you are taking the 61A midterm. There are n questions left on the exam and t time remaining. Completing question i takes i time but gives you $i + 50$ points on the exam. Write a program that will output the highest possible score on the exam given n and t (note: you can't take more than t time).

```
def midterm(n, t):
    """
    >>> midterm(500, 0) # No time left!
    0
    >>> midterm(3, 3) # 51 + 52, questions 1 & 2
    103
    >>> midterm(3, 5) # 52 + 53, questions 2 & 3
    105
    >>> midterm(4, 9) # 52 + 53 + 54, questions 2 & 3 & 4
    159
    """
    if _____:

        return _____

    else:

        return _____

    if n <= 0 or t <= 0:
        return 0
    else:
        return max(n + 50 + midterm(n - 1, t - n), midterm(n - 1, t))
```

Suggested Time: 10 min

- Optional, for if you have time.